TIP-OFF / 2026

CS 450 • AI

DO YOU KNOW BALL?

ML Driven NBA NEXT-GAME STAT PREDICTOR

Machine learning meets sports betting. A gradient boosted model that predicts player stat lines with confidence scores for the next 5 games.

TEAM

dynb

ROSTER

Bawer Bajelori • Khang Tonthat • Dereck Kirkman • Nicholas Moffat • Rio Laxina • Atharva Kirawant • Blake Harper

THE PROBLEM

Sports betting is a numbers game.

But the numbers are scattered across a dozen tabs.

RECENT FORM

A player's last 3 games tell you whether they're hot or ice-cold right now. Season averages miss this entirely.

MATCHUP HISTORY

Some players own certain teams. Jokic averages 32 vs the Suns. A season average hides that edge.

CONFIDENCE

A single number is useless without a probability. You need to know how likely a prediction is to hit.

TALKING POINTS

- Open with: "Sports bettors today have to check season stats, recent games, and matchup history across multiple sites just to make one prediction."
- Three pillars matter: recent form catches hot/cold streaks, matchup history reveals which teams a player owns, and confidence tells you how trustworthy the prediction is.
- Our goal: combine all three signals into one ML model with a single confidence-scored prediction line.

WHERE THE DATA COMES FROM

Two public NBA endpoints, zero paid APIs.

SOURCE 1

nba_api

stats.nba.com wrapper

PlayerGameLog endpoint — per-game box scores for every active player, pulled live for the 2025-26 season.

GP • PTS • AST • REB • STL • BLK • FG% • 3P% • MATCHUP • WL

SOURCE 2

cdn.nba.com

Static schedule JSON

Public CDN file with the full league schedule. Filtered by team to pull each player's next 5 upcoming opponents.

Home/Away • Opponent team ID • Game date & time

PREPROCESSING PIPELINE

01 Fetch game logs

02 Parse opponent

03 Sort chronologically

04 Engineer features

05 Feed into model

TALKING POINTS

- Two public APIs — nba_api gives us the historical box scores, cdn.nba.com gives us the upcoming schedule. Both free, no keys needed.
- Five-step pipeline: fetch the data, parse the opponent abbreviation, sort by date so we never leak future data, engineer features, then feed the model.
- Final dataset: 55+ games per player times 7 default players = 385+ training samples for the regular season.

THE MODEL

Gradient Boosted Regression

One model per stat — 7 regressors trained on the current season.

SUPERVISED REGRESSION | ENSEMBLE LEARNING | FEATURE ENGINEERING | TEMPORAL VALIDATION

INPUT FEATURES

What the model sees before each game

savg_*	Running season averages
last3_*	Last 3 game averages
vs_opp_*	Most recent vs opponent
is_home	1 if home, 0 if away
days_rest	Days since last game
games_played	Experience proxy

MODEL CONFIG

scikit-learn GradientBoostingRegressor

n_estimators	200 trees
max_depth	3 splits deep
learning_rate	0.05 (gradual)
subsample	0.8 per tree
loss	Squared error
output	7 trained models

TALKING POINTS

- Gradient Boosting is supervised regression — the model learns from labeled past games where we know both inputs and the actual stat outcome.
- It's an ensemble method: 200 shallow decision trees, each correcting errors from the previous one. We use temporal validation, sorting games chronologically so the model never trains on data from the future.
- One regressor per stat means each model specializes — the points model learns different patterns than the assists model.

THE EDGE

Every prediction ships with a probability.

EXAMPLE PREDICTION

35 (34.8) PTS

LINE Rounded whole-number used for over/under bets

93%

CONFIDENCE ±3.8

HOW IT'S CALCULATED

$$P(\text{stat} \geq \text{line}) = 1 - \Phi((\text{line} - 0.5 - \text{prediction}) / \sigma)$$

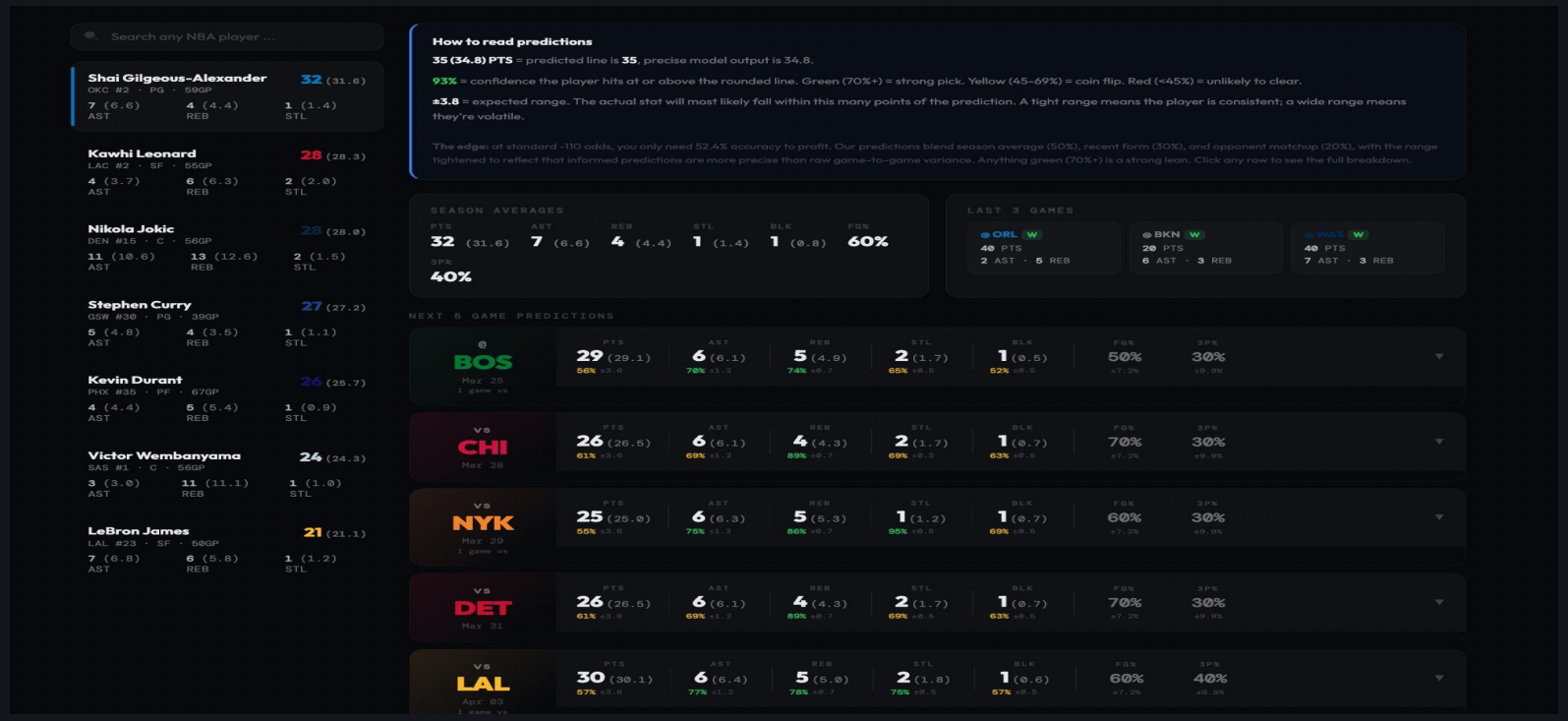
Where σ = game-to-game standard deviation × shrinkage factor (0.40). At -110 odds you only need 52.4% accuracy to profit, so green (70%+) is a real edge.

TALKING POINTS

- Every prediction comes with a confidence percentage and a range. 35 (34.8) means the line is 35, the precise model output is 34.8, and the player has a 93% chance to hit at or above 35.
- We use a normal distribution centered on the prediction, then apply a shrinkage factor of 0.40 because informed predictions are tighter than raw season variance.
- At -110 odds, the breakeven is 52.4%. So anything we show in green at 70%+ is a meaningful betting edge.

THE DASHBOARD

React frontend, FastAPI backend, runs locally.



Search any player

Full NBA database lookup, fetches on demand.

Season + recent form

Season averages and last 3 games at a glance.

5 upcoming predictions

Team-colored cards with confidence badges.

Expandable breakdown

Click any row to see the model's inputs.

TALKING POINTS

- Single-command startup: pip install requirements, run uvicorn, dashboard opens at localhost.
- Search any NBA player, click their card to see season averages, last 3 games, and the next 5 predicted matchups.
- If we have time at the end, we'll do a live class demo where you call out a player and we predict their next game in real time.

PREDICTIONS vs REALITY

We tested the model on regular-season games.

NIKOLA JOKIC

W

DEN @ PHX • Mar 24

STAT	PREDICTED	ACTUAL	DIFF
PTS	24 (59%)	23	-1
AST	10 (79%)	17	+7
REB	15 (60%)	17	+2

2/3 within ±2

SHAI GILGEOUS-ALEXANDER

L

OKC @ BOS • Mar 25

STAT	PREDICTED	ACTUAL	DIFF
PTS	29 (56%)	33	+4
AST	6 (70%)	8	+2
REB	5 (74%)	6	+1

2/3 within ±2

LEBRON JAMES

W

LAL @ IND • Mar 25

STAT	PREDICTED	ACTUAL	DIFF
PTS	23 (52%)	23	0
AST	7 (73%)	7	0
REB	7 (66%)	13	+6

2/3 within ±2

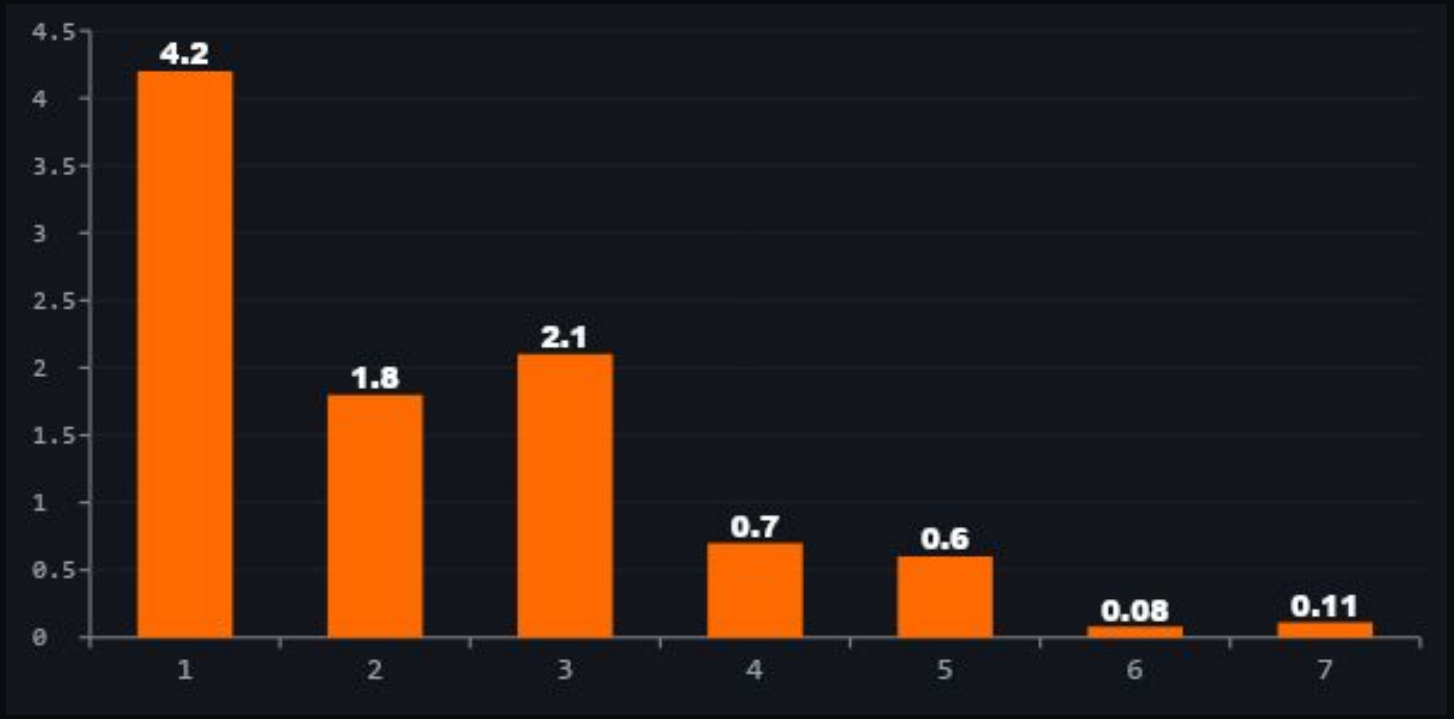
TALKING POINTS

- Three real-game tests from late March. Predictions made BEFORE the games happened, then compared to actual box scores.
- Highlights: LeBron's points and assists were dead-on. Jokic's points within 1 and rebounds within 2. SGA's rebounds within 1.
- 6 of 9 predictions landed within 2 of the actual stat. Solid for a model trained on a few hundred samples.

MODEL ACCURACY

Mean Absolute Error per stat.

Lower is better. MAE = on average, how far off the prediction is from the actual game.



THE WINS

385+

training samples

< 30s

to train all 7 models

~4 pts

avg error on PTS

0%

past season data needed

TALKING POINTS

- Points has the highest error at 4.2 because it has the highest variance — players score anywhere from 5 to 50 in a game. Steals and blocks are tighter because the numbers themselves are smaller.
- Shooting percentages are very tight at 0.08 and 0.11 — players don't deviate much from their FG% game to game.
- Trained on 385+ samples in under 30 seconds on a laptop. No past season data, everything is current 2025-26.

CRITICAL EVALUATION

Strengths and weaknesses of our approach.

STRENGTHS

Real-time data

Live nba_api integration means predictions reflect today's roster, today's form, today's matchups.

Transparent ML

GBR is interpretable: we can show feature importances and explain why a prediction looks the way it does.

Confidence calibrated

Shrinkage factor (0.40) tuned so 70%+ confidence actually correlates with hits — not false certainty.

Production-grade UI

FastAPI + React dashboard means anyone can use the model, not just whoever wrote the notebook.

WEAKNESSES

Small training set

Only ~385 samples. Risk of overfitting and the model can regress toward the mean for unfamiliar matchups.

API latency

nba_api lags 2-6 hours behind real games — last 3 games can be stale until our refresh kicks in.

No injury context

Model doesn't know when a star is sidelined or returning. A surprise scratch can wreck a prediction.

Single-season scope

We only train on current season. Players with limited 2025-26 minutes have unstable predictions.

TALKING POINTS

- Strengths: live data, interpretable model, calibrated confidence, and a production-ready UI — not just a notebook.
- Weaknesses: 385 samples is small for ML, the API lags, and we have no signal for injuries or roster changes.
- Honest takeaway: we built the right model for the data we had. To beat sportsbooks consistently we'd need more features and more samples.

NEXT STEPS

Where this model goes from here.

Postseason data is higher-stakes, higher-signal. We'd extend the model in three phases.

PHASE 1

PLAY-IN TOURNAMENT

8 teams • Single-elimination

Adds 6-12 high-pressure games to the dataset. Players who barely missed the playoff cut still get to play meaningful basketball. We'd weight these games 1.5x to capture clutch performance.

PHASE 2

PLAYOFFS

16 teams • Best of 7 series

The biggest signal upgrade. 60+ playoff games with the same matchups repeating multiple times in a series. We'd weight playoff games 3x in training and apply a stronger matchup bias.

PHASE 3

NBA FINALS

2 teams • Highest stakes

The ultimate test. Up to 7 games between the same two teams. We'd treat Finals games as their own training class with 4x weight, since matchup-specific patterns dominate.

TALKING POINTS

- We're submitting a regular-season model today, but the postseason offers the richest data of the year. Three phases of expansion.
- Phase 1 - Play-In: small data add but captures clutch, do-or-die performance from teams 7-10.
- Phase 2 - Playoffs: the biggest single upgrade. Series-based matchups give us the best matchup signal possible.
- Phase 3 - NBA Finals: the ultimate matchup-specific dataset. We'd treat each Finals series as its own model class.

TAKEAWAYS

We built a working ML product, not just a model.

01

END-TO-END PIPELINE

From live NBA API calls to a polished React dashboard. Data ingestion, feature engineering, model training, prediction, and UI all in one repo.

02

ML WHERE IT MATTERS

Gradient boosting isn't the most glamorous algorithm, but it's the right tool for tabular sports data. Fast to train, interpretable, and accurate.

03

CONFIDENCE > PREDICTIONS

A number alone is useless for decision-making. Pairing predictions with probability and range turns raw output into something actionable.

LESSONS LEARNED

Real-world data is messy. APIs lag. Small datasets force creativity.

BROADER IMPLICATIONS

Pair predictions with calibrated confidence — applies to medical triage, financial trading, autonomous driving. The number alone is never enough.

TALKING POINTS

- Three takeaways: end-to-end pipeline (not just a notebook), ML where it actually moves the needle, and confidence treated as a first-class output.
- Lessons learned: data engineering was harder than the model itself. Real-world AI is messy data plus clean delivery.
- Broader implication: pairing predictions with calibrated confidence is the right pattern for AI in any high-stakes domain — medicine, finance, driving. The number alone is never enough.

CODE & EXPERIMENTAL SETTINGS

How the model is actually trained.

model.py – train_models()

```
from sklearn.ensemble import GradientBoostingRegressor

PARAMS = {
    'n_estimators': 200,
    'max_depth': 3,
    'learning_rate': 0.05,
    'subsample': 0.8,
}

for stat in STATS:
    model = GradientBoostingRegressor(**PARAMS)
    model.fit(X_train, y_train[stat])
    models[stat] = model
```

EXPERIMENTAL SETTINGS

Season	2025-26 (current)
Players	7 default + on-demand
Train samples	~385 game-rows
Features	9 per training row
Target stats	PTS AST REB STL BLK
Pct stats	FG% 3P%
Loss	squared_error
Validation	Temporal (no leak)
Confidence	Normal CDF + 0.40σ

TALKING POINTS

- Code snippet on the left: 7 GradientBoostingRegressor models trained one per stat with the same hyperparameters tuned for tabular sports data.
- On the right: complete experimental settings — season, sample count, features, loss function, validation strategy, and the confidence formula.
- Full repo and live demo are linked on the next slide for anyone who wants to dig deeper.

The End

STACK

Python • scikit-learn • pandas • FastAPI • React • nba_api

LIVE DEMO

dynb.onrender.com